

A Course Based Project Report on
**RIDE DISPATCH SIMULATION
SYSTEM**

Submitted to the
Department of Computer Science and Engineering
in partial fulfilment of the requirements for the completion of
course SOFTWARE ENGINEERING LABORATORY
(22PC1CS201)

**BACHELOR OF
TECHNOLOGY IN
COMPUTER SCIENCE ENGINEERING**

Submitted by

D. DEEKSHITH	24071A05P4
D. ROHITH REDDY	24071A05P6
D. PRANITH REDDY	24071A05P7
M. SRIRAM	24071A05R9
Y. YASHASH CHANDRA	24071A05V4

**Under the Guidance
of Mrs. S. Jahnavi
Ms. Srijitha Majumder**



**VALLURUPALLI NAGESWARA RAO VIGNANA
JYOTHI INSTITUTE OF ENGINEERING AND
TECHNOLOGY**

An Autonomous Institute, NAAC Accredited with 'A++' Grade
NBA Accredited for CE, EEE, ME, ECE, CSE, EIE, IT B.Tech. Programs Approved by
AICTE, New Delhi, Affiliated to JNTU-H, Recognized as "College with Potential for

Excellence” by UGC Vignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O),
Hyderabad TS 500 090 India

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade
NBA Accredited for CE, EEE, ME, ECE, CSE, EIE, IT B.Tech. Programmes

Approved by AICTE, New Delhi, Affiliated to JNTU-H, Recognized as "College with
Potential for Excellence" by UGC Vignana Jyothi Nagar, Pragathi Nagar, Nizampet
(S.O), Hyderabad TS 500 090 India



TAMASOMA JYOTIRGAMAYA

CERTIFICATE

This is to certify that **D. Deekshith (24071A05P4), D. Rohith Reddy (24071A05P6), D. Pranith Reddy (24071A05P7), M. Sriram (24071A05R9), Y. Yashash Chandra (24071A05V4)**, have successfully completed their Course Based Project work at **Computer Science & Engineering Department** of **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology**, Hyderabad entitled "**RIDE DISPATCH SIMULATION SYSTEM**" in partial fulfilment of the requirements for the award of **B.Tech** during the academic year 2025-2026.

**Mrs. S.
Jahnavi**
Assistant
Professor
CSE
Department
VNR VJIE

**Ms. Srijitha
Majumder**
Assistant Professor
CSE Department
VNR VJIE

Dr. V Baby
Associate Professor &
HOD
CSE Department
VNR VJIE

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade
NBA Accredited for CE, EEE, ME, ECE, CSE, EIE, IT B.Tech. Programmes

Approved by AICTE, New Delhi, Affiliated to JNTU-H, Recognized as "College
with Potential for Excellence" by UGC Vignana Jyothi Nagar, Pragathi Nagar,
Nizampet (S.O), Hyderabad



TAMASOMA JYOTIRGAMAYA

DECLARATION

This is to certify that the project work entitled "**RIDE DISPATCH SIMULATION SYSTEM**" submitted in VNR Vignana Jyothi Institute of Engineering & Technology in partial fulfilment of requirement for the award of Bachelor of Technology in Computer Science and Engineering is a Bonafide report of the work carried out by us under the guidance and supervision of **Mrs. S. Jahnavi**, Assistant Professor, Department of CSE, VNRVJIET and **Ms. Srijitha Majumder**, Assistant Professor, Department of CSE, VNRVJIET. To the best of our knowledge, this report has not been submitted in any form to any university or institution for the award of any degree or diploma.

Submission Dt:

D. Deekshith **D. Rohith Reddy**
D. Pranith

Reddy

M. Sriram

Chandra

Y. Yashash

24071A05P4	24071A05P6	24071A05P7	24071A05R9
	24071A05V4	II B.Tech-CSE	II B.Tech-CSE II B.Tech-
CSE	II B.Tech-CSE	II B.Tech-CSE VNR VJIET	VNR VJIET
	VNR VJIET	VNR VJIET	VNR VJIET

ACKNOWLEDGEMENT

An endeavor over a long period can be successful only with the advice and support of many well-wishers. We take this opportunity to express our gratitude and appreciation to all of them.

We are indebted to our venerable principal **Dr. B Chennakesava Rao** for this unflinching devotion, which led us to complete this project. The support, encouragement given by him and his motivation lead us to complete this project.

We wish to express our profound gratitude to **Dr. V Baby, Associate Professor** and **HOD CSE Department, VNR Vignana Jyothi Institute of Engineering and Technology** for their constant and dedicated service to brighten our career. With great pleasure we express our gratitude to the faculty advisors, **Mrs. S. Jahnavi** and **Ms. Srijitha Majumder** for their timely help, constant guidance, cooperation, support and encouragement throughout this project.

Finally, we wish to express our deep sense of gratitude and sincere thanks to our parents, friends and all our well-wishers who have technically and non-technically contributed for the successful completion of our course-based project.

D. Deekshith (24071A05P4)

D. Rohith Reddy (24071A05P6)

D. Pranith Reddy (24071A05P7)

M. Sriram (24071A05R9)

Y. Yashash Chandra (24071A05V4)

Table of Contents

S.NO.	CONTENTS	PAGE NO.
1.	INTRODUCTION	1
2.	OBJECTIVES	2
3.	LITERATURE REVIEW	3-4
4.	EXISTING SYSTEM	5-6
5.	METHODOLOGY	7-8
6.	ARCHITECTURE AND IMPLEMENTATION	9-11
7.	PROGRAM CODE	12-15
8.	CONCLUSIONS	16
9.	FUTURE SCOPE	17-18
10.	REFERENCES	19

1.INTRODUCTION

The Ride Dispatch Simulator is a real-time simulation project developed using Python and Tkinter GUI to demonstrate the practical implementation of Operating System scheduling and resource management concepts in a real-world ride-hailing environment. The project models riders as processes and drivers as resources, while the dispatch system acts as the scheduler responsible for allocating resources efficiently.

The main objective of this project is to simulate how ride requests are managed, scheduled, and executed using different CPU scheduling algorithms such as First Come First Serve (FCFS), Priority Scheduling, Shortest Job First (SJF), and Round Robin Scheduling. These algorithms are used to determine which rider should be served first and how drivers are assigned dynamically based on availability and proximity.

The simulator provides a graphical interface that visually represents riders, drivers, ride execution, waiting queues, and system performance metrics. It also includes advanced operating system concepts such as preemption, aging, starvation prevention, turnaround time, throughput, and resource utilization. The system continuously updates through a tick-based simulation mechanism, making it possible to observe how scheduling decisions affect system efficiency in real time.

Additionally, the project includes visual components such as a city map, Gantt chart, rider status table, statistics dashboard, and event log, allowing users to clearly understand the internal working of scheduling algorithms and dispatch operations. The nearest available driver is automatically assigned to a rider, optimizing ride allocation and reducing waiting time.

Overall, this project successfully combines theoretical Operating System concepts with practical simulation and visualization techniques, making it an effective educational and analytical tool for understanding scheduling

algorithms and resource allocation in modern transportation systems.

2. OBJECTIVES

2.1 Primary Objectives

1. Simulate Operating System scheduling concepts — Demonstrate how CPU scheduling algorithms such as FCFS, Priority Scheduling, SJF, and Round Robin can be applied in a real-world ride dispatch environment.
2. Efficient ride allocation system — Automatically assign riders to the nearest available drivers to reduce waiting time and improve resource utilization.
3. Real-time ride dispatch simulation — Build a tick-based simulation engine that continuously updates rider states, driver availability, and scheduling decisions dynamically.
4. Visualize scheduling behavior graphically — Provide an interactive graphical interface using Tkinter to display riders, drivers, execution flow, Gantt charts, and system statistics in real time.
5. Implement multiple scheduling algorithms — Enable comparison and analysis of different scheduling techniques by integrating FCFS, Priority Scheduling, SJF, and Round Robin algorithms into a single system.

2.2 Secondary Objectives

1. Starvation prevention using aging — Increase the priority of long-waiting riders dynamically to ensure fair execution and prevent indefinite waiting.
2. Preemption support in Round Robin scheduling — Interrupt and re-queue incomplete rides after a fixed time quantum to simulate time-sharing systems.
3. Performance metric analysis — Calculate and display important operating system metrics such as average waiting time, turnaround time, throughput, and driver utilization.
4. Nearest-driver optimization — Use distance calculation logic to identify and assign the closest available driver for efficient ride dispatching.
5. Interactive event monitoring — Maintain a real-time event log to track rider arrivals, driver assignments, ride completions, and

scheduling operations for debugging and analysis.

6. Graphical representation of execution flow — Use a Gantt chart and live city map visualization to clearly represent process execution and resource allocation.
7. Configurable simulation environment — Allow users to modify scheduling algorithms, simulation speed, and auto-spawn functionality for experimentation and learning purposes.

3. LITERATURE REVIEW

3.1 Operating System Scheduling Algorithms

CPU scheduling is one of the most important functions of an Operating System. It determines which process should be executed by the CPU at a given time in order to maximize efficiency, reduce waiting time, and improve overall system performance. Different scheduling algorithms are designed to achieve different objectives such as fairness, responsiveness, and throughput.

The scheduling algorithms implemented in this project are:

Algorithm	Description
FCFS (First Come First Serve)	Processes are executed in the order they arrive in the queue.
Priority Scheduling	Processes with higher priority are executed first.
SJF (Shortest Job First)	Processes with the shortest execution time are executed first.
Round Robin	Each process receives a fixed time quantum in cyclic order.

FCFS is simple and easy to implement but may suffer from the convoy effect, where shorter tasks wait behind long-running tasks. Priority Scheduling improves responsiveness for important tasks but may lead to starvation if low-priority processes never get CPU time. SJF minimizes average waiting time but requires knowledge of execution duration. Round Robin improves fairness and responsiveness by sharing CPU time equally among processes.

This project applies these scheduling techniques to a ride dispatch system where riders behave as processes and drivers behave as system resources.

3.2 Resource Allocation and Dispatch Systems

Resource allocation refers to the efficient distribution of limited system resources among competing processes. In modern ride-hailing systems such as Uber and Ola, drivers act as resources that must be allocated

efficiently to rider requests.

The dispatch mechanism in this project follows a nearest-resource allocation approach. When a rider request arrives, the system searches for the nearest available driver and assigns the ride accordingly. This strategy minimizes rider waiting time and improves service efficiency.

The project also simulates dynamic resource states:

- Available drivers
- Busy drivers
- Running rides
- Waiting riders

This closely resembles resource allocation and process management techniques used in Operating Systems.

3.3 Real-Time Simulation Systems

Real-time simulations are systems that continuously update their internal state over time to imitate real-world behavior. In this project, a tick-based simulation mechanism is used, where each tick represents a unit of system time.

At every simulation tick, the following operations are performed:

- Rider generation
- Queue scheduling
- Driver assignment
- Ride execution
- Ride completion
- Statistics update

This event-driven simulation approach allows users to observe how scheduling algorithms affect system performance dynamically and visually.

3.4 Aging and Starvation Prevention

One major issue in priority-based scheduling is starvation, where low-priority processes may wait indefinitely while higher-priority processes continue to execute.

To solve this problem, Operating Systems use a technique called aging. Aging gradually increases the priority of processes that remain in the

waiting queue for a long period of time.

This project implements aging by increasing the priority of riders after a specified waiting threshold. This ensures fairness and prevents indefinite waiting.

The system also tracks starvation conditions by monitoring riders whose waiting time exceeds a predefined threshold.

3.5 Graphical Visualization of Scheduling

Visualization techniques are widely used in Operating System education to understand scheduling behavior and process execution. Gantt charts are commonly used to represent process timelines, execution duration, and waiting periods.

This project includes several graphical visualization components:

- City map visualization
- Rider and driver tracking
- Gantt chart representation
- Statistics dashboard
- Event logging system

The graphical interface is developed using Python's Tkinter library, which provides real-time updates and interactive simulation controls.

3.6 Python and Tkinter for Simulation Development

Python is a high-level programming language widely used for simulation, automation, and GUI development because of its readability and extensive standard libraries.

The project uses:

- tkinter for graphical user interface development
- math for distance calculations
- random for generating riders and driver positions
- time for simulation timing

Tkinter provides an efficient framework for building interactive desktop applications and visual simulations without requiring external dependencies.

3.7 Related Work

Several systems and educational simulators exist for demonstrating CPU scheduling algorithms and dispatch systems.

Examples include:

- CPU Scheduling Simulators used in Operating System laboratories
- Ride-sharing dispatch systems used in transportation applications
- Traffic simulation systems
- Resource management simulators

However, most traditional scheduling simulators focus only on textual execution without real-world visualization. Similarly, many ride-dispatch systems do not explicitly demonstrate Operating System concepts such as preemption, aging, starvation handling, and scheduling algorithm comparison.

This project uniquely combines:

- Operating System scheduling
- Resource allocation
- Real-time simulation
- Interactive graphical visualization
- Ride dispatch modeling

into a single integrated educational system.

4. EXISTING SYSTEMS

4.1 Traditional CPU Scheduling Simulators

Most traditional Operating System scheduling simulators focus only on theoretical process execution using text-based outputs or simple console interfaces. These simulators mainly demonstrate scheduling algorithms such as FCFS, SJF, Priority Scheduling, and Round Robin without incorporating any real-world application context.

Although useful for understanding algorithm logic, these systems have several limitations:

- Limited visualization — Most simulators display only tables or numerical outputs without graphical representation.
- No real-world analogy — Processes are represented abstractly, making it difficult for users to relate scheduling concepts to practical applications.
- Static execution — Many systems execute predefined processes rather than dynamically generated tasks.
- Limited interaction — Users cannot observe real-time scheduling behaviour visually.

4.2 Basic Ride Dispatch Systems

Simple ride dispatch systems are commonly used in academic mini-projects to demonstrate driver allocation and ride management. These systems typically perform straightforward ride assignments based on driver availability.

However, such systems usually lack advanced Operating System concepts and scheduling mechanisms.

Their major limitations include:

- No scheduling algorithm support — Ride assignment is generally based only on arrival order.
- No preemption handling — Running rides cannot be interrupted or rescheduled.
- No starvation prevention — Long-waiting riders may remain unserved indefinitely.
- Minimal analytical features — Performance metrics such as waiting time

and throughput are often absent.

4.3 Queue-Based Dispatch Approaches

Several systems implement simple queue-based dispatch mechanisms where ride requests are stored and processed sequentially.

Although queue-based systems are easy to implement, they suffer from several disadvantages:

- Poor optimization — Nearest-driver allocation may not be considered.
- Higher waiting times — Requests are processed sequentially regardless of urgency.
- Lack of fairness — Important or urgent requests may experience unnecessary delays.
- No dynamic scheduling — Systems cannot adapt based on workload or rider priority.

4.4 GUI-Based Educational Simulators

Some modern educational simulators provide graphical interfaces for process scheduling demonstrations. These systems improve understanding through animations and charts.

Despite these improvements, many GUI simulators still face limitations:

- Focus only on CPU processes — No real-world dispatch application is included.
- Limited interactivity — Users often cannot dynamically generate processes during execution.
- No resource allocation modeling — Resource management concepts are not fully visualized.
- Restricted scalability — Simulations are usually limited to small datasets and fixed scenarios.

4.5 Comparative Analysis

The table below compares existing systems with the proposed Ride Dispatch Simulator.

Criterion	Traditional Scheduling	Basic Ride Dispatch	Queue-Based	Proposed System
-----------	------------------------	---------------------	-------------	-----------------

	Simulators	Systems	System s	
Real-time Simulation	✗	Partial	✗	✓
Multiple Scheduling Algorithms	✓	✗	✗	✓
Graphical Visualization	Partial	Partial	✗	✓
Resource Allocation	Partial	✓	✓	✓
Preemption Support	Partial	✗	✗	✓
Aging & Starvation Handling	✗	✗	✗	✓
Dynamic Rider Generation	✗	Partial	✗	✓
Performance Metrics	Partial	✗	✗	✓
Interactive Controls	Partial	Partial	✗	✓
Real-World OS Mapping	✗	Partial	✗	✓

The proposed Ride Dispatch Simulator combines Operating System scheduling concepts, real-time simulation, resource allocation, and graphical visualization into a single integrated system, overcoming the limitations of existing approaches.

5. METHODOLOGY

5.1 Development Approach

The project was developed using an incremental and modular approach, where each component was implemented and tested individually before full integration into the complete system.

1. Layer 1 — Core Scheduling Engine:

The Ride System class was developed first to implement the core scheduling logic, rider management, driver allocation, queue handling, and simulation timing.

2. Layer 2 — Scheduling Algorithm Integration:

Multiple scheduling algorithms including FCFS, Priority Scheduling, SJF, and Round Robin were implemented and validated independently using sample rider requests.

3. Layer 3 — Resource Allocation Mechanism:

The nearest-driver allocation logic was integrated to ensure efficient assignment of available drivers to rider requests based on distance calculations.

4. Layer 4 — GUI Development:

The Tkinter graphical interface was developed to visualize riders, drivers, scheduling execution, Gantt charts, statistics, and event logs in real time.

5. Layer 5 — System Integration and Testing:

All modules were combined and tested together to verify scheduling behaviour, simulation flow, visualization accuracy, and performance metric calculations.

5.2 Scheduling Algorithm Selection

Four scheduling algorithms were selected to analyze different process scheduling behaviours within the ride dispatch environment.

- FCFS (First Come First Serve) — Processes rider requests in the exact order of arrival using FIFO queue management.
- Priority Scheduling — Gives preference to riders with higher priority values, simulating urgent or VIP ride requests.

- SJF (Shortest Job First) — Selects riders with the shortest ride distance first to minimize overall waiting time.
- Round Robin Scheduling — Allocates a fixed execution quantum to each rider and re-queues incomplete rides to simulate preemptive multitasking.

Using multiple scheduling algorithms allows comparison of fairness, efficiency, responsiveness, and resource utilization.

5.3 Tick-Based Simulation Mechanism

The system uses a time-driven simulation model where each simulation cycle is represented as a “tick.”

During every tick, the following operations are executed:

1. Rider generation and queue updates
2. Aging and starvation handling
3. Scheduling queue sorting
4. Ride execution progress
5. Driver assignment and release
6. Statistics and visualization updates

This approach provides real-time simulation behaviour and enables continuous observation of scheduling decisions.

5.4 Driver Allocation Strategy

The project implements a nearest-driver allocation mechanism to optimize dispatch efficiency.

The system calculates the Euclidean distance between riders and available drivers using coordinate-based calculations. The nearest free driver is assigned to the rider request.

This strategy:

- Reduces rider waiting time
- Improves driver utilization
- Simulates real-world dispatch optimization

5.5 Aging and Starvation Prevention

Priority-based scheduling may lead to starvation, where low-priority riders remain waiting indefinitely.

To address this issue, the project implements an aging mechanism:

- Riders waiting beyond a threshold automatically receive increased priority.
- Starvation conditions are detected using waiting-time thresholds.

This ensures fairness and balanced execution across all riders.

5.6 Process Execution Model

Each rider request behaves as an independent process in the simulation.

The execution model includes:

- Waiting state
- Running state
- Preempted state
- Completed state

In Round Robin scheduling, riders exceeding the time quantum are preempted and returned to the waiting queue for future execution.

This process model accurately simulates Operating System process scheduling behaviour.

5.7 Visualization Methodology

The graphical user interface was developed using Python's Tkinter library to provide real-time visual representation of the system.

The interface includes:

1. City Map Visualization — Displays riders and drivers dynamically using graphical objects.
2. Gantt Chart — Represents waiting time and execution duration of rides.
3. Statistics Dashboard — Displays performance metrics such as waiting time, turnaround time, throughput, and utilization.
4. Event Log System — Records scheduling events, assignments, ride completions, and algorithm changes.

The visualization layer improves understanding of scheduling and resource

allocation behaviour.

5.8 Testing Methodology

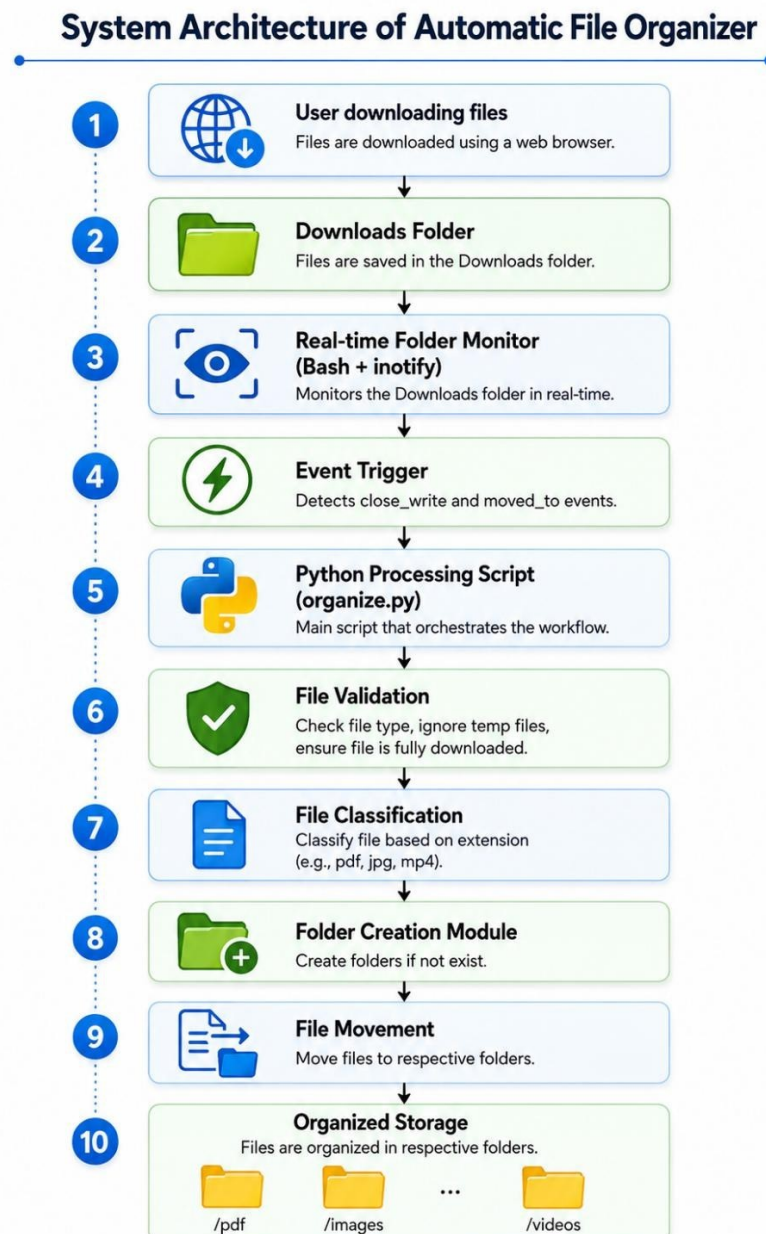
The project was validated through multiple test scenarios to ensure correctness and reliability.

1. Testing different scheduling algorithms individually.
2. Verifying nearest-driver assignment logic.
3. Testing Round Robin preemption and queue re-entry.
4. Monitoring aging and starvation prevention mechanisms.
5. Evaluating statistics such as waiting time and throughput.
6. Performing GUI validation for map rendering, Gantt chart updates, and event logging.

These tests confirmed accurate scheduling execution and real-time system behaviour.

6. SYSTEM ARCHITECTURE AND IMPLEMENTATION

6.1 Architectural Overview



6.2 Directory Structure

file-organizer/

├─ organizer.py # Python worker: classifies and moves files

├─ watcher.sh # Bash event listener: invokes

organizer assets/

├─ logs.png # Screenshot: journalctl live logs

├─ output.png # Screenshot: organized directory structure

└─ service-status.png # Screenshot: systemd

service status README.md

6.3 systemd Unit Configuration

The service unit file (/etc/systemd/system/file-organizer.service) is configured with the parameters described below.

Parameter	Value	Purpose
Type	simple	Foreground process model; systemd tracks the direct process
User / Group	ubuntu	Runs under a non-privileged user for security
ExecStart	/bin/bash watcher.sh	Launches the inotify event listener
Restart	always	Automatically restarts the service on any exit
RestartSec	3	3-second delay before each restart

		attempt
StandardOutput	journal	Routes stdout to systemd's journal
StandardError	journal	Routes stderr to systemd's journal
PYTHONUNBUFFERED	1	Flushes Python log output in real time

6.4 Data Flow

1. A file is written or moved into /home/ubuntu/down/.
2. The Linux kernel's inotify subsystem emits a close_write or moved_to event.
3. inotifywait (running inside watcher.sh) receives the event and outputs the filename.
4. watcher.sh constructs the full file path and spawns organizer.py as a background process.
5. organizer.py extracts the file extension, checks it against the transient-file ignore list, polls until the file size is stable, creates the target subdirectory if it does not yet exist, and moves the file.
6. Success or error messages are written to stdout/stderr and captured by journald.

6.5 Resulting Output Structure

After processing, the watched directory self-organizes into the following layout:

/home/ubuntu/down/

```

├─ pdf/
|   └─ report.pdf
|   └─ invoice.pdf

```

```
|— jpg/  
|   └─ photo.jpg  
|— mp4/  
|   └─ video.mp4  
└─ txt/  
    └─ notes.txt
```

7. SOURCE CODE

7.1 ride_dispatch_simulator.py

"""

Ride Dispatch Simulator — OS Course Project

=====

Maps Operating System scheduling & resource-management concepts
to a real-world ride-hailing dispatch scenario.

OS Concept → Ride Analogy

Process → Rider (ride request)

Resource → Driver (vehicle)

Scheduler → Dispatch Algorithm

Aging → Priority boost for long-waiting riders

Starvation → Rider waiting beyond threshold

Scheduling Algorithms Implemented:

1. FCFS — First-Come, First-Served
2. Priority — Highest priority dispatched first (with aging)
3. SJF — Shortest Job First (by estimated ride distance)
4. Round Robin — Time-quantum based; incomplete rides re-queued

"""

import tkinter as tk

from tkinter import ttk, font as tkfont

import random

import math

import time as _time

#

CONSTANTS

#

MAP_W, MAP_H = 520, 400

GANTT_W, GANTT_H = 440, 140

RIDE_DURATION = 4 # ticks for a full ride

RR_QUANTUM = 2 # Round-Robin time quantum

AGING_THRESHOLD = 3 # ticks before aging kicks in

STARVATION_THRESHOLD = 8 # ticks to flag starvation

NUM_DRIVERS = 5

MAX_COORD = 50 # map logical coordinate range

— Colours —

BG = "#1a1a2e"

```

BG_PANEL    = "#16213e"
BG_CARD     = "#0f3460"
ACCENT      = "#e94560"
ACCENT2     = "#533483"
TEXT        = "#eaeaea"
TEXT_DIM    = "#8892b0"
GREEN       = "#00e676"
AMBER       = "#ffab00"
RED         = "#ff5252"
BLUE        = "#448aff"
GRAY        = "#546e7a"
CYAN        = "#00e5ff"
PURPLE      = "#b388ff"

```

```

STATE_COLORS = {
    "waiting": RED,
    "assigned": AMBER,
    "running": GREEN,
    "preempted": PURPLE,
    "done": GRAY,
}

```

```

ALGO_COLORS = {
    "FCFS": "#448aff",
    "Priority": "#e94560",
    "SJF": "#00e676",
    "Round Robin": "#ffab00",
}

```

```
#
```

```
# DATA CLASSES
```

```
#
```

```
class Rider:
```

```
    """Represents a process (ride request)."""
```

```
    _id_counter = 0
```

```

    def __init__(self, x=None, y=None, dest_x=None, dest_y=None,
priority=None):
        Rider._id_counter += 1
        self.rid = Rider._id_counter
        self.x = x if x is not None else random.randint(2, MAX_COORD - 2)
        self.y = y if y is not None else random.randint(2, MAX_COORD - 2)
        self.dest_x = dest_x if dest_x is not None else random.randint(2,
MAX_COORD - 2)
        self.dest_y = dest_y if dest_y is not None else random.randint(2,

```

```

MAX_COORD - 2)
    self.priority = priority if priority is not None else random.randint(1, 5)
    self.base_priority = self.priority
    self.wait = 0                # ticks in queue
    self.state = "waiting"
    self.arrival_tick = 0        # set when added
    self.start_tick = None       # when ride starts
    self.end_tick = None        # when ride completes
    self.assigned_driver = None
    self.ride_distance = self._calc_dist()
    self.remaining = RIDE_DURATION # remaining service ticks
    self.total_service = RIDE_DURATION

```

```

def _calc_dist(self):
    return round(math.sqrt((self.dest_x - self.x)**2 +
                          (self.dest_y - self.y)**2), 1)

```

```

@classmethod
def reset_counter(cls):
    cls._id_counter = 0

```

```

class Driver:
    """Represents a resource (driver/vehicle)."""
    def __init__(self, did, x=None, y=None):
        self.did = did
        self.x = x if x is not None else random.randint(2, MAX_COORD - 2)
        self.y = y if y is not None else random.randint(2, MAX_COORD - 2)
        self.home_x = self.x
        self.home_y = self.y
        self.free = True
        self.busy_ticks = 0
        self.total_ticks = 0
        self.current_rider = None

```

```

#

```

```

# SIMULATION ENGINE
#

```

```

class RideSystem:
    def __init__(self):
        self.tick_count = 0
        self.riders = []      # all riders ever
        self.queue = []       # waiting queue
        self.running = []     # (rider, driver, start_tick)

```

```

self.completed = []
self.preempted = []      # Round-Robin preempted list
self.drivers = [Driver(i + 1) for i in range(NUM_DRIVERS)]
self.scheduler = "FCFS"
self.auto_spawn = False
self.auto_spawn_interval = 3
self.log = []           # event log entries

def _log(self, msg):
    self.log.append(f"[T={self.tick_count:>3}] {msg}")

def reset(self):
    Rider.reset_counter()
    self.__init__()

# — Distance helper —
@staticmethod
def dist(x1, y1, x2, y2):
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

# — Add a rider —
def add_rider(self):
    r = Rider()
    r.arrival_tick = self.tick_count
    self.riders.append(r)
    self.queue.append(r)
    self._log(f"Rider R{r.rid} arrived (prio={r.priority},
dist={r.ride_distance})")
    return r

# — Aging —
def _aging(self):
    for r in self.queue:
        r.wait += 1
        if r.wait > AGING_THRESHOLD and r.priority < 10:
            r.priority += 1
            self._log(f"Rider R{r.rid} aged → priority {r.priority}")

# — Schedule / sort the queue —
def _schedule(self):
    if self.scheduler == "Priority":
        self.queue.sort(key=lambda r: -r.priority)
    elif self.scheduler == "SJF":
        self.queue.sort(key=lambda r: r.ride_distance)
    # FCFS & Round Robin keep arrival order (FIFO)

# — Find nearest free driver to a rider —

```

```

def _nearest_driver(self, rider):
    best = None
    best_dist = float('inf')
    for d in self.drivers:
        if d.free:
            dd = self.dist(d.x, d.y, rider.x, rider.y)
            if dd < best_dist:
                best_dist = dd
                best = d
    return best

# — Assign riders from queue to free drivers —
def _assign(self):
    assigned = []
    for r in list(self.queue):
        d = self._nearest_driver(r)
        if d is None:
            break
        d.free = False
        d.current_rider = r
        r.state = "running"
        r.assigned_driver = d.did
        if r.start_tick is None:
            r.start_tick = self.tick_count
        self.running.append((r, d, self.tick_count))
        assigned.append(r)
        self._log(f"Rider R{r.rid} → Driver D{d.did}"
(algo={self.scheduler}))
    for r in assigned:
        self.queue.remove(r)

# — Complete / progress rides —
def _progress(self):
    still_running = []
    for r, d, start in self.running:
        r.remaining -= 1
        d.busy_ticks += 1

        if r.remaining <= 0:
            # ride complete
            d.free = True
            d.current_rider = None
            r.state = "done"
            r.end_tick = self.tick_count
            self.completed.append(r)
            self._log(f"Rider R{r.rid} completed (wait={r.wait},
turnaround={r.end_tick - r.arrival_tick})")

```

```

else:
    # Round Robin preemption check
    if self.scheduler == "Round Robin":
        elapsed = self.tick_count - start
        if elapsed >= RR_QUANTUM and r.remaining > 0:
            # preempt
            d.free = True
            d.current_rider = None
            r.state = "preempted"
            r.assigned_driver = None
            self.queue.append(r) # re-queue
            self._log(f'Rider R{r.rid} preempted (RR quantum),
remaining={r.remaining}')
            continue
            still_running.append((r, d, start))

self.running = still_running

# — Auto-spawn —
def _auto_spawn(self):
    if self.auto_spawn and self.tick_count % self.auto_spawn_interval ==
0:
        self.add_rider()

# — Tick —
def tick(self):
    self.tick_count += 1
    for d in self.drivers:
        d.total_ticks += 1
    self._auto_spawn()
    self._aging()
    self._schedule()
    self._progress()
    self._assign()

# — Stats —
def stats(self):
    completed = self.completed
    n = len(completed)
    if n == 0:
        return {
            "avg_wait": 0, "avg_turnaround": 0,
            "throughput": 0, "utilization": 0,
            "starvation": 0, "completed": 0,
            "in_queue": len(self.queue), "running": len(self.running),
        }
    avg_wait = sum(r.wait for r in completed) / n

```



```

    avg_ta = sum((r.end_tick - r.arrival_tick) for r in completed) / n
    throughput = n / max(self.tick_count, 1)
    total_busy = sum(d.busy_ticks for d in self.drivers)
    total_total = sum(d.total_ticks for d in self.drivers)
    utilization = (total_busy / max(total_total, 1)) * 100
    starvation = sum(1 for r in self.queue if r.wait >
STARVATION_THRESHOLD)
    return {
        "avg_wait": round(avg_wait, 2),
        "avg_turnaround": round(avg_ta, 2),
        "throughput": round(throughput, 3),
        "utilization": round(utilization, 1),
        "starvation": starvation,
        "completed": n,
        "in_queue": len(self.queue),
        "running": len(self.running),
    }

```

```
#
```

```
# TOOLTIP HELPER
```

```
#
```

```

class ToolTip:
    def __init__(self, widget, text=""):
        self.widget = widget
        self.text = text
        self.tip_window = None
        widget.bind("<Enter>", self.show)
        widget.bind("<Leave>", self.hide)

    def show(self, event=None):
        if self.tip_window or not self.text:
            return
        x = self.widget.winfo_rootx() + 20
        y = self.widget.winfo_rooty() + 20
        self.tip_window = tw = tk.Toplevel(self.widget)
        tw.wm_overrideredirect(True)
        tw.wm_geometry(f'x+{x}y+{y}')
        label = tk.Label(tw, text=self.text, justify="left",
                        background="#222", foreground=TEXT,
                        relief="solid", borderwidth=1,
                        font=("Consolas", 9), padx=6, pady=4)
        label.pack()

    def hide(self, event=None):

```

```

        if self.tip_window:
            self.tip_window.destroy()
            self.tip_window = None

#
=====
# GUI APP
#
=====
class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("⚙ Ride Dispatch Simulator — OS Scheduling")
        self.geometry("1200x780")
        self.minsize(1080, 700)
        self.configure(bg=BG)
        self.system = RideSystem()
        self.sim_running = False
        self.speed = 700          # ms per tick
        self._after_id = None

        self._setup_styles()
        self._build_ui()

# — ttk Dark Theme —
def _setup_styles(self):
    style = ttk.Style(self)
    style.theme_use("clam")

    style.configure(".", background=BG, foreground=TEXT,
                    fieldbackground=BG_PANEL, borderwidth=0)
    style.configure("TFrame", background=BG)
    style.configure("Card.TFrame", background=BG_CARD)
    style.configure("TLabel", background=BG, foreground=TEXT,
                    font=("Segoe UI", 10))
    style.configure("Header.TLabel", background=BG, foreground=TEXT,
                    font=("Segoe UI", 18, "bold"))
    style.configure("Sub.TLabel", background=BG,
foreground=TEXT_DIM,
                    font=("Segoe UI", 9))
    style.configure("Stat.TLabel", background=BG_CARD,
foreground=CYAN,
                    font=("Consolas", 13, "bold"))
    style.configure("StatTitle.TLabel", background=BG_CARD,
foreground=TEXT_DIM,
                    font=("Segoe UI", 8))

```

```

# Buttons
style.configure("Accent.TButton", background=ACCENT,
foreground="white",
                font=("Segoe UI", 10, "bold"), padding=(14, 6))
style.map("Accent.TButton",
         background=[("active", "#c0392b"), ("disabled", GRAY)])

style.configure("TButton", background=BG_CARD, foreground=TEXT,
                font=("Segoe UI", 10), padding=(12, 6))
style.map("TButton",
         background=[("active", ACCENT2), ("disabled", GRAY)])

# Combobox
style.configure("TCombobox", fieldbackground=BG_PANEL,
                background=BG_CARD, foreground=TEXT,
                selectbackground=ACCENT2, selectforeground=TEXT)

# Treeview
style.configure("Treeview", background=BG_PANEL,
foreground=TEXT,
                fieldbackground=BG_PANEL, rowheight=26,
                font=("Consolas", 10))
style.configure("Treeview.Heading", background=BG_CARD,
                foreground=CYAN, font=("Segoe UI", 10, "bold"))
style.map("Treeview", background=[("selected", ACCENT2)])

# Scale
style.configure("TScale", background=BG, troughcolor=BG_PANEL)

# Radiobutton / Checkbutton
style.configure("TRadiobutton", background=BG, foreground=TEXT,
                font=("Segoe UI", 10))
style.configure("TCheckbutton", background=BG, foreground=TEXT,
                font=("Segoe UI", 10))

# — Build entire UI —
def _build_ui(self):
    # — HEADER —
    hdr = ttk.Frame(self)
    hdr.pack(fill="x", padx=16, pady=(12, 4))

    ttk.Label(hdr, text="⚙ Ride Dispatch Simulator",
              style="Header.TLabel").pack(side="left")

    self.clock_label = ttk.Label(hdr, text="Tick: 0", style="Sub.TLabel")
    self.clock_label.pack(side="right", padx=12)

```

```

sep = tk.Frame(self, bg=ACCENT, height=2)
sep.pack(fill="x", padx=16)

# — MAIN PANE —
main = ttk.Frame(self)
main.pack(fill="both", expand=True, padx=12, pady=6)

left = ttk.Frame(main)
left.pack(side="left", fill="both", expand=True)

right = ttk.Frame(main)
right.pack(side="right", fill="both", padx=(8, 0))

# — LEFT: MAP —
map_frame = ttk.Frame(left)
map_frame.pack(fill="both", expand=True, pady=(0, 6))

map_title = ttk.Frame(map_frame)
map_title.pack(fill="x")
ttk.Label(map_title, text="□ City Map", font=("Segoe UI", 11, "bold"),
          foreground=CYAN, background=BG).pack(side="left")

self.canvas = tk.Canvas(map_frame, width=MAP_W, height=MAP_H,
                        bg="#0d1b2a", highlightthickness=1,
                        highlightbackground=BG_CARD)
self.canvas.pack(fill="both", expand=True, pady=(4, 0))
self._draw_grid()

# — LEFT: GANTT —
gantt_frame = ttk.Frame(left)
gantt_frame.pack(fill="x", pady=(0, 2))

ttk.Label(gantt_frame, text="□ Gantt Chart", font=("Segoe UI", 11,
"bold"),
          foreground=CYAN, background=BG).pack(anchor="w")

self.gantt = tk.Canvas(gantt_frame, width=GANTT_W,
height=GANTT_H,
                        bg="#0d1b2a", highlightthickness=1,
                        highlightbackground=BG_CARD)
self.gantt.pack(fill="x", expand=True, pady=(4, 0))

# — RIGHT: TABLE —
tbl_frame = ttk.Frame(right)
tbl_frame.pack(fill="both", expand=True, pady=(0, 6))

```

```

        ttk.Label(tbl_frame, text="⏏ Rider Table", font=("Segoe UI", 11,
"bold"),
                foreground=CYAN, background=BG).pack(anchor="w")

        cols = ("ID", "State", "Prio", "Wait", "Driver", "Dist", "Rem")
        self.tree = ttk.Treeview(tbl_frame, columns=cols, show="headings",
height=10)
        widths = {"ID": 45, "State": 75, "Prio": 45, "Wait": 45,
                "Driver": 55, "Dist": 50, "Rem": 40}
        for c in cols:
            self.tree.heading(c, text=c)
            self.tree.column(c, width=widths.get(c, 55), anchor="center",
minwidth=35)
        self.tree.pack(fill="both", expand=True, pady=(4, 0))

        # — RIGHT: STATS —
        stats_frame = ttk.Frame(right, style="Card.TFrame")
        stats_frame.pack(fill="x", pady=(0, 6), ipady=8)

        ttk.Label(stats_frame, text="⏏ Statistics", font=("Segoe UI", 11,
"bold"),
                foreground=CYAN, background=BG_CARD).pack(anchor="w",
padx=8, pady=(4, 2))

        sg = ttk.Frame(stats_frame, style="Card.TFrame")
        sg.pack(fill="x", padx=8)

        self.stat_labels = {}
        stat_items = [
            ("Avg Wait", "avg_wait"), ("Avg Turnaround", "avg_turnaround"),
            ("Throughput", "throughput"), ("Utilization %", "utilization"),
            ("Starvation", "starvation"), ("Completed", "completed"),
            ("In Queue", "in_queue"), ("Running", "running"),
        ]
        for i, (title, key) in enumerate(stat_items):
            r, c = divmod(i, 4)
            cell = ttk.Frame(sg, style="Card.TFrame")
            cell.grid(row=r, column=c, padx=6, pady=3, sticky="nsew")
            sg.columnconfigure(c, weight=1)
            ttk.Label(cell, text=title, style="StatTitle.TLabel").pack()
            lbl = ttk.Label(cell, text="0", style="Stat.TLabel")
            lbl.pack()
            self.stat_labels[key] = lbl

        # — RIGHT: EVENT LOG —
        log_frame = ttk.Frame(right)
        log_frame.pack(fill="both", expand=True)

```

```

        ttk.Label(log_frame, text="□ Event Log", font=("Segoe UI", 11,
"bold"),
                foreground=CYAN, background=BG).pack(anchor="w")

        self.log_text = tk.Text(log_frame, height=6, bg="#0d1b2a",
fg=TEXT_DIM,
                                font=("Consolas", 9), wrap="word",
                                state="disabled", relief="flat",
                                insertbackground=TEXT,
selectbackground=ACCENT2)
        log_scroll = ttk.Scrollbar(log_frame, command=self.log_text.yview)
        self.log_text.configure(yscrollcommand=log_scroll.set)
        log_scroll.pack(side="right", fill="y")
        self.log_text.pack(fill="both", expand=True, pady=(4, 0))

        # — BOTTOM: CONTROLS —
        ctrl_bar = ttk.Frame(self)
        ctrl_bar.pack(fill="x", padx=16, pady=(4, 12))

        sep2 = tk.Frame(self, bg=ACCENT, height=2)
        sep2.pack(fill="x", padx=16, before=ctrl_bar)

        # Row 1: buttons
        row1 = ttk.Frame(ctrl_bar)
        row1.pack(fill="x", pady=(6, 2))

        self.btn_add = ttk.Button(row1, text="□ Add Rider",
                                command=self._add_rider, style="Accent.TButton")
        self.btn_add.pack(side="left", padx=3)
        ToolTip(self.btn_add, "Spawn a new rider at a random location")

        self.btn_start = ttk.Button(row1, text="▶ Start",
                                command=self._toggle_sim)
        self.btn_start.pack(side="left", padx=3)
        ToolTip(self.btn_start, "Start / Pause the simulation")

        self.btn_step = ttk.Button(row1, text="□ Step",
                                command=self._step)
        self.btn_step.pack(side="left", padx=3)
        ToolTip(self.btn_step, "Advance exactly one tick")

        self.btn_reset = ttk.Button(row1, text="□ Reset",
                                command=self._reset)
        self.btn_reset.pack(side="left", padx=3)
        ToolTip(self.btn_reset, "Reset entire simulation")

```

```

# Scheduler
ttk.Label(row1, text=" Scheduler:",
foreground=TEXT_DIM).pack(side="left", padx=(16, 4))
self.sched_var = tk.StringVar(value="FCFS")
sched_cb = ttk.Combobox(row1, textvariable=self.sched_var,
                        values=["FCFS", "Priority", "SJF", "Round Robin"],
                        state="readonly", width=13)
sched_cb.pack(side="left", padx=3)
sched_cb.bind("<<ComboboxSelected>>", self._change_sched)
ToolTip(sched_cb, "Select scheduling algorithm")

self.sched_badge = ttk.Label(row1, text=" FCFS ",
                             background=ALGO_COLORS["FCFS"],
                             foreground="white",
                             font=("Consolas", 10, "bold"))
self.sched_badge.pack(side="left", padx=6)

# Auto-spawn
self.auto_var = tk.BooleanVar(value=False)
auto_cb = ttk.Checkbutton(row1, text="Auto-Spawn",
variable=self.auto_var,
                        command=self._toggle_auto)
auto_cb.pack(side="left", padx=(16, 3))
ToolTip(auto_cb, "Auto-add a rider every 3 ticks")

# Speed
ttk.Label(row1, text=" Speed:",
foreground=TEXT_DIM).pack(side="left", padx=(16, 4))
self.speed_var = tk.IntVar(value=700)
speed_scale = ttk.Scale(row1, from_=100, to=2000,
variable=self.speed_var,
                        orient="horizontal", length=120,
                        command=self._change_speed)
speed_scale.pack(side="left", padx=3)
self.speed_lbl = ttk.Label(row1, text="700ms",
foreground=TEXT_DIM)
self.speed_lbl.pack(side="left")
ToolTip(speed_scale, "Simulation speed (ms per tick)")

# — Legend —
legend = ttk.Frame(ctrl_bar)
legend.pack(fill="x", pady=(4, 0))
items = [("● Waiting", RED), ("● Running", GREEN),
        ("● Preempted", PURPLE), ("● Done", GRAY),
        ("◆ Driver Free", BLUE), ("◆ Driver Busy", AMBER)]
for txt, color in items:
    ttk.Label(legend, text=txt, foreground=color,

```

```

        font=("Segoe UI", 9)).pack(side="left", padx=8)

# — Draw grid on canvas —
def _draw_grid(self):
    for i in range(0, MAX_COORD + 1, 10):
        x = self._mx(i)
        y = self._my(i)
        self.canvas.create_line(x, 0, x, MAP_H, fill="#1a2a3a", width=1)
        self.canvas.create_line(0, y, MAP_W, y, fill="#1a2a3a", width=1)

def _mx(self, lx):
    return int(lx / MAX_COORD * (MAP_W - 20)) + 10

def _my(self, ly):
    return int(ly / MAX_COORD * (MAP_H - 20)) + 10

# =====
# CALLBACKS
# =====
def _add_rider(self):
    self.system.add_rider()
    self._refresh()

def _toggle_sim(self):
    self.sim_running = not self.sim_running
    self.btn_start.config(text="⏸ Pause" if self.sim_running else "▶ Start")
    if self.sim_running:
        self._run_loop()

def _step(self):
    if not self.sim_running:
        self.system.scheduler = self.sched_var.get()
        self.system.tick()
        self._refresh()

def _reset(self):
    self.sim_running = False
    if self._after_id:
        self.after_cancel(self._after_id)
        self._after_id = None
    self.system.reset()
    self.btn_start.config(text="▶ Start")
    self._refresh()

def _change_sched(self, event=None):
    algo = self.sched_var.get()
    self.system.scheduler = algo

```



```

        color = ALGO_COLORS.get(algo, BLUE)
        self.sched_badge.config(text=f" {algo} ", background=color)
        self.system._log(f"Scheduler changed → {algo}")
        self._refresh()

def _toggle_auto(self):
    self.system.auto_spawn = self.auto_var.get()

def _change_speed(self, val=None):
    self.speed = int(float(self.speed_var.get()))
    self.speed_lbl.config(text=f"{self.speed}ms")

# =====
# SIMULATION LOOP
# =====
def _run_loop(self):
    if not self.sim_running:
        return
    self.system.scheduler = self.sched_var.get()
    self.system.tick()
    self._refresh()
    self._after_id = self.after(self.speed, self._run_loop)

# =====
# REFRESH ALL VIEWS
# =====
def _refresh(self):
    self._refresh_clock()
    self._refresh_table()
    self._refresh_map()
    self._refresh_stats()
    self._refresh_gantt()
    self._refresh_log()

def _refresh_clock(self):
    self.clock_label.config(text=f"Tick: {self.system.tick_count} | "
                               f"Riders: {len(self.system.riders)} | "
                               f"Algo: {self.system.scheduler}")

def _refresh_table(self):
    for row in self.tree.get_children():
        self.tree.delete(row)
    # Show running + queue + preempted first, then completed (last 10)
    active = [r for r in self.system.riders if r.state != "done"]
    done = list(reversed(self.system.completed[-10:]))
    for r in active + done:
        drv = f"D{r.assigned_driver}" if r.assigned_driver else "—"

```

```

        rem = r.remaining if r.state == "running" else "-"
        tag = r.state
        self.tree.insert("", "end",
                        values=(f'R{r.ride}', r.state.capitalize(),
                                r.priority, r.wait, drv,
                                r.ride_distance, rem),
                        tags=(tag,))
    # Color rows
    for state, color in STATE_COLORS.items():
        self.tree.tag_configure(state, foreground=color)

def _refresh_map(self):
    self.canvas.delete("obj")

    # Draw connection lines for active rides
    for r, d, _ in self.system.running:
        x1, y1 = self._mx(d.x), self._my(d.y)
        x2, y2 = self._mx(r.x), self._my(r.y)
        self.canvas.create_line(x1, y1, x2, y2, fill=GREEN,
                                width=1, dash=(4, 4), tags="obj")

    # Draw riders
    for r in self.system.riders:
        if r.state == "done":
            continue
        rx, ry = self._mx(r.x), self._my(r.y)
        color = STATE_COLORS.get(r.state, RED)
        self.canvas.create_oval(rx - 6, ry - 6, rx + 6, ry + 6,
                                fill=color, outline="white", width=1, tags="obj")
        self.canvas.create_text(rx, ry - 12, text=f'R{r.ride}',
                                fill=TEXT, font=("Consolas", 8), tags="obj")

    # Destination marker
    if r.state in ("waiting", "running", "preempted"):
        dx, dy = self._mx(r.dest_x), self._my(r.dest_y)
        self.canvas.create_line(rx, ry, dx, dy,
                                fill=color, width=1, dash=(2, 3),
                                stipple="gray25", tags="obj")
        self.canvas.create_rectangle(dx - 3, dy - 3, dx + 3, dy + 3,
                                    fill=color, outline="", tags="obj")

    # Draw drivers
    for d in self.system.drivers:
        dx, dy = self._mx(d.x), self._my(d.y)
        color = BLUE if d.free else AMBER
        # Diamond shape
        pts = [dx, dy - 9, dx + 9, dy, dx, dy + 9, dx - 9, dy]
        self.canvas.create_polygon(pts, fill=color, outline="white",

```

```

        width=1, tags="obj")
    self.canvas.create_text(dx, dy - 15, text=f"D{d.did}",
        fill=color, font=("Consolas", 8, "bold"), tags="obj")

def _refresh_stats(self):
    s = self.system.stats()
    for key, lbl in self.stat_labels.items():
        val = s.get(key, 0)
        lbl.config(text=str(val))
        # Color starvation red if > 0
        if key == "starvation" and val > 0:
            lbl.config(foreground=RED)
        elif key == "starvation":
            lbl.config(foreground=CYAN)

def _refresh_gantt(self):
    self.gantt.delete("all")
    riders = self.system.riders
    if not riders:
        self.gantt.create_text(GANTT_W // 2, GANTT_H // 2,
            text="No riders yet", fill=TEXT_DIM,
            font=("Segoe UI", 10))

    return

    max_tick = max(self.system.tick_count, 1)
    # Show at most 15 riders
    display_riders = riders[-15:]
    bar_h = max(8, min(18, (GANTT_H - 30) // max(len(display_riders),
1)))
    left_margin = 45
    right_margin = 10
    chart_w = GANTT_W - left_margin - right_margin
    top_margin = 20

    # Time axis
    self.gantt.create_line(left_margin, top_margin - 4,
        left_margin + chart_w, top_margin - 4,
        fill=TEXT_DIM, width=1)
    step = max(1, max_tick // 10)
    for t in range(0, max_tick + 1, step):
        x = left_margin + int(t / max_tick * chart_w)
        self.gantt.create_text(x, 8, text=str(t), fill=TEXT_DIM,
            font=("Consolas", 7))
        self.gantt.create_line(x, top_margin - 6, x, top_margin - 2,
            fill=TEXT_DIM)

    for i, r in enumerate(display_riders):

```

```

y = top_margin + i * (bar_h + 2)
# Label
self.gantt.create_text(5, y + bar_h // 2, text=f"R{r.riid}",
                      fill=TEXT_DIM, font=("Consolas", 7), anchor="w")

# Waiting bar (red)
arr = r.arrival_tick
start = r.start_tick if r.start_tick else self.system.tick_count
x1 = left_margin + int(arr / max_tick * chart_w)
x2 = left_margin + int(start / max_tick * chart_w)
if x2 > x1:
    self.gantt.create_rectangle(x1, y, x2, y + bar_h,
                              fill=RED, outline="", tags="gantt_bar")

# Running bar (green)
if r.start_tick is not None:
    end = r.end_tick if r.end_tick else self.system.tick_count
    x1 = left_margin + int(r.start_tick / max_tick * chart_w)
    x2 = left_margin + int(end / max_tick * chart_w)
    if x2 > x1:
        self.gantt.create_rectangle(x1, y, x2, y + bar_h,
                                    fill=GREEN, outline="", tags="gantt_bar")

def _refresh_log(self):
    self.log_text.config(state="normal")
    self.log_text.delete("1.0", "end")
    # Show last 50 entries
    for entry in self.system.log[-50:]:
        self.log_text.insert("end", entry + "\n")
    self.log_text.see("end")
    self.log_text.config(state="disabled")

#
#
# MAIN
#
#
if __name__ == "__main__":
    app = App()
    app.mainloop()

```

7.2 Code Walkthrough

The Ride Dispatch Simulator is developed using Python and Tkinter to demonstrate Operating System scheduling and resource allocation concepts in a real-world ride dispatch environment. The system is mainly divided into three major sections: data classes, simulation engine, and graphical user interface.

The Rider class represents a ride request and behaves like a process in an Operating System. Each rider is assigned a unique ID, pickup coordinates, destination coordinates, priority value, waiting time, and execution state. The class also calculates ride distance using the Euclidean distance formula, which is later used in the Shortest Job First (SJF) scheduling algorithm. The rider object stores additional execution information such as arrival time, remaining execution time, assigned driver, and completion status.

The Driver class represents a system resource similar to a CPU or hardware resource in Operating Systems. Each driver contains information such as driver ID, location coordinates, availability status, busy time, and currently assigned rider. The driver state changes dynamically between free and busy during ride execution.

The RideSystem class acts as the core simulation engine. It manages all riders, drivers, scheduling operations, execution flow, and statistical analysis. The class maintains separate lists for waiting riders, currently running rides, completed rides, and preempted rides. It also stores the currently selected scheduling algorithm and event logs for monitoring system activities.

The `add_rider()` function creates new rider objects and inserts them into the waiting queue. Every rider arrival is recorded in the event log along with priority and ride distance information. This function simulates process creation in Operating Systems.

The `_aging()` function prevents starvation in priority scheduling. During every simulation tick, the waiting time of riders in the queue is increased. If a rider waits beyond a predefined threshold, the system automatically increases its priority value. This ensures fairness and prevents low-priority riders from waiting indefinitely.

The `_schedule()` function applies different scheduling algorithms to the rider queue. In Priority Scheduling, riders are sorted according to priority values. In SJF, riders are sorted according to ride distance. FCFS and Round Robin scheduling preserve the original arrival order using FIFO queue behavior.

The `_nearest_driver()` function is responsible for resource allocation. It calculates the distance between riders and all available drivers and selects the nearest free driver for assignment. This minimizes rider waiting time and improves overall dispatch efficiency.

The `_assign()` function assigns riders from the waiting queue to free drivers. Once assigned, the rider state changes to “running,” and the driver becomes busy. The assignment details are added to the running queue and recorded in the system log.

The `_progress()` function manages ride execution. During every simulation tick, the remaining execution time of active rides decreases. When the remaining time becomes zero, the ride is marked as completed, and the assigned driver becomes available again. In Round Robin scheduling, if the execution exceeds the configured time quantum, the rider is preempted and returned to the waiting queue for future execution.

The `tick()` function controls the complete simulation cycle. Every tick performs sequential operations including rider generation, aging, scheduling, execution progress, and driver assignment. This function acts as the heartbeat of the simulation engine and continuously updates the entire system state.

The `stats()` function calculates important performance metrics such as average waiting time, turnaround time, throughput, utilization, starvation count, and completed rides. These metrics help analyze and compare the efficiency of different scheduling algorithms.

The graphical user interface is implemented using the `App` class and `Tkinter` library. The interface contains multiple visualization components including a city map, rider table, Gantt chart, statistics panel, and event log. The GUI also provides controls for adding riders, starting or pausing the simulation, selecting scheduling algorithms, adjusting speed, and resetting the system.

The `_refresh_map()` function visually displays riders and drivers on the city map. Riders are represented using colored circles, while drivers are displayed as diamond-shaped objects. Active ride assignments are shown using dashed lines between drivers and riders. The `_refresh_gantt()` function generates a Gantt chart showing rider waiting time and execution duration, helping users understand scheduling behavior visually.

Finally, the main block initializes the `Tkinter` application and starts the simulation using `app.mainloop()`. This launches the complete graphical Ride Dispatch Simulator and enables real-time interaction with the scheduling system.

8. CONCLUSION

The Ride Dispatch Simulator project demonstrates a practical and interactive application of Operating System scheduling and resource management concepts in a real-world ride-hailing environment. The following conclusions are drawn from the design, implementation, and testing of the system:

1. **Scheduling algorithms significantly influence system performance.** By implementing FCFS, Priority Scheduling, SJF, and Round Robin algorithms, the project clearly demonstrates how different scheduling strategies affect waiting time, turnaround time, throughput, fairness, and resource utilization.
2. **Real-time simulation improves understanding of Operating System concepts.** The tick-based execution model allows continuous observation of process scheduling, driver allocation, preemption, and queue management, making theoretical concepts easier to visualize and analyze.
3. **Graphical visualization enhances learning and analysis.** The integration of a city map, Gantt chart, rider table, statistics dashboard, and event log provides a clear visual representation of scheduling behavior, execution flow, and system performance in real time.
4. **Resource allocation optimization improves dispatch efficiency.** The nearest-driver assignment mechanism minimizes rider waiting time and increases driver utilization, demonstrating the importance of efficient resource management techniques in distributed systems.
5. **Aging effectively prevents starvation.** The implementation of priority aging ensures fairness among riders by gradually increasing the priority of long-waiting requests, preventing indefinite delays in execution.
6. **Round Robin scheduling successfully demonstrates preemption.** The use of time quantum and ride re-queuing accurately simulates multitasking and time-sharing behavior found in modern Operating Systems.
7. **Performance metrics provide analytical insight.** Metrics such as

average waiting time, turnaround time, throughput, starvation count, and utilization allow direct comparison of scheduling algorithms and help evaluate system efficiency under different workloads.

8. **Modular design improves maintainability and scalability.**

Separating the project into scheduling, resource management, simulation, and visualization components allows easier testing, debugging, and future expansion of the system.

9. **The project has strong educational and practical value.** It not only demonstrates core Operating System principles but also models real-world ride dispatch operations, making it useful for both academic learning and simulation-based analysis.

10. **In conclusion,** the Ride Dispatch Simulator is an efficient, interactive, and visually rich simulation system that successfully combines Operating System scheduling algorithms, resource allocation strategies, real-time execution, and graphical visualization into a single integrated application.

9. FUTURE SCOPE

The current implementation of the Ride Dispatch Simulator provides a stable and efficient platform for demonstrating Operating System scheduling and resource allocation concepts. The following future developments are identified as high-value extensions for the project.

9.1 Real-Time GPS and Map Integration

The current system uses simulated coordinate-based positioning for riders and drivers. Future versions can integrate real-world map services such as Google Maps API or OpenStreetMap to provide live route visualization, real-time distance calculation, and realistic navigation support. This would significantly improve the practical applicability of the dispatch system.

9.2 Database Integration

At present, rider and driver data are stored temporarily in memory during simulation execution. Future implementations can integrate databases such as MySQL, PostgreSQL, or MongoDB to enable persistent storage of ride history, driver records, scheduling statistics, and system logs. This would support long-term analytics and data management.

9.3 Multi-Threaded Simulation Engine

The current simulation follows a single-threaded execution model. Future enhancements can introduce multi-threading or multiprocessing techniques to simulate concurrent rider requests, parallel driver execution, and large-scale real-time dispatch scenarios. This would improve scalability and better represent modern distributed systems.

9.4 AI-Based Smart Scheduling

Future versions can incorporate Artificial Intelligence and Machine Learning techniques for intelligent ride scheduling and prediction. AI-based algorithms can optimize driver allocation, estimate ride demand, predict traffic conditions, and dynamically adjust scheduling strategies to improve efficiency and reduce waiting times.

9.5 Dynamic Traffic and Road Conditions

The present system assumes static movement conditions. Future implementations can simulate dynamic traffic density, road congestion, accidents, and weather conditions to create a more realistic transportation environment. This would allow deeper analysis of scheduling performance under varying real-world conditions.

9.6 Web-Based Deployment

The current project is implemented as a desktop application using Tkinter. Future development can convert the system into a web-based platform using technologies such as React, Node.js, Flask, or Django. A web deployment would improve accessibility, scalability, and remote monitoring capabilities.

9.7 Advanced Scheduling Algorithms

Currently, the simulator supports FCFS, Priority Scheduling, SJF, and Round Robin algorithms. Future versions can implement advanced scheduling techniques such as Multilevel Queue Scheduling, Multilevel Feedback Queue Scheduling, Earliest Deadline First (EDF), and Hybrid Scheduling models for deeper Operating System analysis.

9.8 Real-Time Analytics Dashboard

A more advanced analytics dashboard can be developed to provide graphical reports, comparative scheduling analysis, rider demand trends, heat maps, and driver performance metrics. This would improve monitoring and support decision-making through data visualization.

9.9 User Authentication and Driver Profiles

Future versions can include login systems for riders and drivers along with profile management features. Driver ratings, ride history, earnings, and rider feedback systems can also be integrated to simulate commercial ride-hailing platforms more realistically.

9.10 Cloud and Distributed System Support

The project can be extended into a distributed dispatch architecture using cloud platforms and microservices. Multiple dispatch servers can cooperate to handle large-scale ride requests, enabling study of distributed scheduling, fault tolerance, and load balancing concepts.

9.11 Mobile Application Integration

Future enhancements can include Android or iOS mobile applications that interact with the dispatch system in real time. Riders can request rides directly through the app, while drivers can receive ride notifications and navigation updates dynamically.

10. REFERENCES

1. Python Tkinter Documentation — Python GUI Library Documentation.
<https://docs.python.org/3/library/tkinter.html>
2. Python ttk Module Documentation — Tk Themed Widgets Documentation.
<https://docs.python.org/3/library/tkinter.ttk.html>
3. Python math Module Documentation — Python 3 Standard Library.
<https://docs.python.org/3/library/math.html>
4. Python random Module Documentation — Python 3 Standard Library.
<https://docs.python.org/3/library/random.html>
5. Python time Module Documentation — Python 3 Standard Library.
<https://docs.python.org/3/library/time.html>
6. Operating System Concepts — Silberschatz, A., Galvin, P. B., & Gagne, G.
Wiley Publications. Chapters on CPU Scheduling and Process Management.
7. Stallings, W. (2018). Operating Systems: Internals and Design Principles.
Pearson Education. Chapters on Scheduling Algorithms and Resource Allocation.
8. GeeksforGeeks — CPU Scheduling Algorithms Explained.
<https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/>
9. Python Official Documentation — Object-Oriented Programming Concepts.
<https://docs.python.org/3/tutorial/classes.html>
10. TkDocs — Modern Tkinter GUI Development Guide.
<https://tkdocs.com/>
11. Round Robin Scheduling Algorithm — Operating System Scheduling Reference.

<https://www.geeksforgeeks.org/round-robin-scheduling-in-operating-system/>

12. Shortest Job First (SJF) Scheduling — Operating System Reference.

<https://www.geeksforgeeks.org/shortest-job-first-or-sjf-cpu-scheduling/>

13. Priority Scheduling Algorithm — Operating System Reference.

<https://www.geeksforgeeks.org/priority-scheduling-in-operating-system/>

14. FCFS Scheduling Algorithm — Operating System Reference.

<https://www.geeksforgeeks.org/program-for-fcfs-cpu-scheduling-set-1/>

15. Python Canvas Widget Documentation — Tkinter Graphics and Visualization.

<https://anzelg.github.io/rin2/book2/2405/docs/tkinter/canvas.html>